

NAME - ADITYA KUMAR

UNIVERSITY ROLL NUMBER - 34230821034

Course Name: Soft Computing Lab

Course Code: PECAIML693

3rd Year 6th Semester

QUESTION 1

Implement Union, Intersection, Complement, Algebraic Difference, Algebraic product, Multiplication of fuzzy set with crisp number, Power of fuzzy set, Algebraic sum, Bounded sum, Bounded difference and Cartesian product on fuzzy sets.

```
def union(a,b):
    res=[max(a[i],b[i]) for i in range(len(a))]
    return res
a=[0.3,0.8,0.1]
b=[0.5,0.7,0.2]
print("UNION OF a, b",union(a,b))
➤ UNION OF a, b [0.5, 0.8, 0.2]
```

Union

```
def intersection(a,b):
    res=[min(a[i],b[i]) for i in range(len(a))]
    return res
a=[0.3,0.8,0.1]
b=[0.5,0.7,0.2]
print("INTERSECTION OF a, b",intersection(a,b))
➤ INTERSECTION OF a, b [0.3, 0.7, 0.1]
```

Intersection,

```
def complement(a):
    res=[x:=float(1-a[i]) for i in range(len(a))]
    return res
a=[0.3,0.6,0.1]
print("Complement of a",complement(a))
➤ Complement of a [0.7, 0.4, 0.9]
```

Complement

```
def difference(a,b):
    res=[(intersection(a,complement(b)))]
    return res
a=[0.2,0.8,0.1]
b=[0.5,0.4,0.2]
print("DIFFERENCE of a,b",difference(a,b))
DIFFERENCE of a,b [[0.2, 0.6, 0.1]]
```

Algebraic Difference

```
def scalar_product(alpha,a):
    res=[alpha*a[i] for i in range(len(a))]
    return res
a=[0.2,0.8,0.1]
print("SCALAR PRODUCT of a,b",scalar_product(2,a))
➤ SCALAR PRODUCT of a,b [0.4, 1.6, 0.2]
```

Multiplication of fuzzy set with crisp Number

```
def fuzzy_product(a,b):
    res=[a[i]*b[i]for i in range(len(a))]
    return res
a=[0.3,0.5,0.5]
b=[0.5,0.4,0.4]
print("FUZZY PRODUCT OF a, b", fuzzy_product(a,b))
```

Algebraic product

➤ FUZZY PRODUCT OF a, b [0.15, 0.2, 0.2]

```
import itertools
def power_set(s):
    return list(itertools.chain.from_iterable(itertools.combinations(s, r) for r in
range(len(s))))
my_set = {0.5,0.4,0.2}
my_power_set = power_set(my_set)
print(my_power_set)
```

➤ [(), (0.5,), (0.4,), (0.2,), (0.5, 0.4), (0.5, 0.2), (0.4, 0.2)]

```
def bound_diff(a,b):
    res=[max(0,((a[i]+b[i])-1)) for i in range(len(a))]
    return res
a=[0.2,0.8,0.1] # Bounded difference
b=[0.5,0.4,0.2]
print("BOUND DIFFERENCE of a,b",bound_diff(a,b))
➤ BOUND DIFFERENCE of a,b [0, 0.200000000000000018, 0]
```

```
def dis_sum(a,b):
    res=[union((intersection(complement(a),b)),intersection(a,complement(b)))]
    return res
a=[0.2,0.8,0.1] # Algebraic sum
b=[0.5,0.4,0.2]
print(dis_sum(a,b))
➤ [[0.5, 0.6, 0.2]]
```

```
def bound_sum(a,b):
    res=[min(1,((a[i]+b[i]))) for i in range(len(a))]
    return res
a=[0.2,0.8,0.1] # Bounded sum
b=[0.5,0.4,0.2]
print("BOUND SUM of a,b",bound_sum(a,b))
BOUND SUM of a,b [0.7, 1, 0.30000000000000004]
```

```
def bound_diff(a,b):
    res=[max(0,((a[i]+b[i])-1)) for i in range(len(a))]
    return res
a=[0.2,0.8,0.1] # Bounded difference
b=[0.5,0.4,0.2]
print("BOUND DIFFERENCE of a,b",bound_diff(a,b))
BOUND DIFFERENCE of a,b [0, 0.200000000000000018, 0]
```

```
import numpy as np
def cartesian_product(a,b):
    k=[]
    for i in range(len(a)):
        res=[min(a[i],b[j]) for j in range(len(b))]
        k.append(res)
    return k
a=[0.2,0.3,0.5,0.6]
b=[0.8,0.6,0.3]
#print(np.array(cartesian_product(a,b)))
z=cartesian_product(a,b)
for i in z:
    print(i)
[0.2, 0.2, 0.2]
[0.3, 0.3, 0.3]
[0.5, 0.5, 0.3]
[0.6, 0.6, 0.3]
```

QUESTION 2

Realization and plotting of different fuzzy membership function: Triangular Membership function, Trapezoidal membership function, Sigmoid membership function, Generalized bell membership function and Gaussian membership function.

```
import numpy as np
import matplotlib.pyplot as plt

# Triangular Membership Function
def triangular(x, a, b, c):
    return np.maximum(0, np.minimum((x - a) / (b - a), (c - x) / (c - b)))

# Trapezoidal Membership Function
def trapezoidal(x, a, b, c, d):
    return np.maximum(0, np.minimum(np.minimum((x - a) / (b - a), 1, (d - x) / (d - c)), 1))

# Sigmoid Membership Function
def sigmoid(x, a, b):
    return 1 / (1 + np.exp(-a*(x-b)))

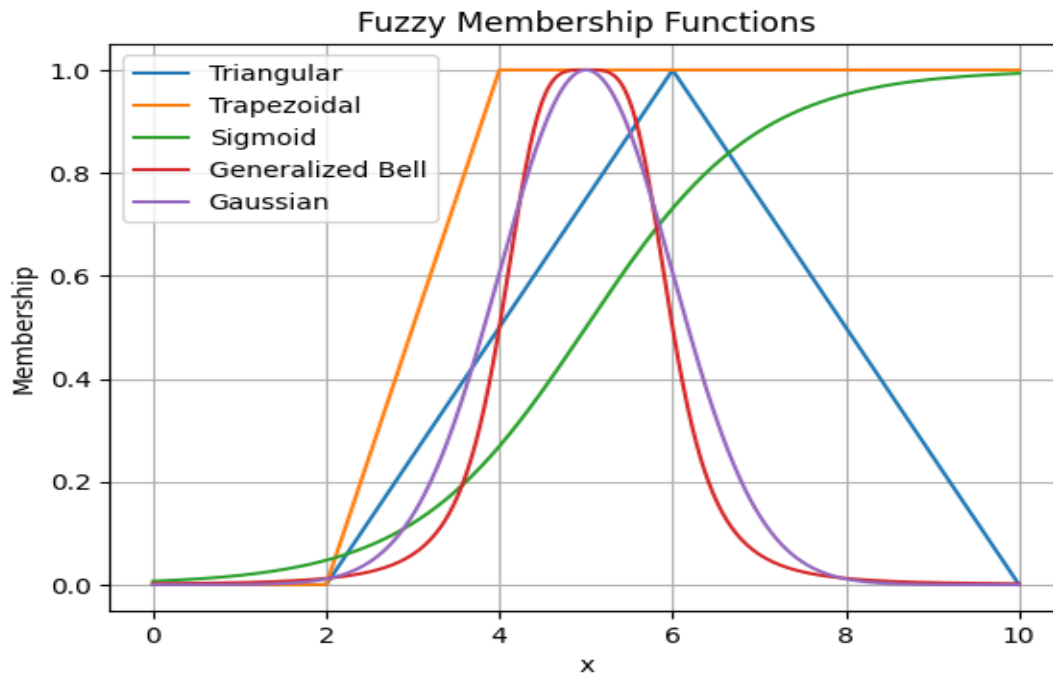
# Generalized Bell Membership Function
def bell(x, a, b, c):
    return 1 / (1 + np.abs((x - c) / a) ** (2 * b))

# Gaussian Membership Function
def gaussian(x, mean, sigma):
    return np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

# Plotting the membership functions
x = np.linspace(0, 10, 1000)

# Triangular
plt.plot(x, triangular(x, 2, 6, 10), label='Triangular')
# Trapezoidal
plt.plot(x, trapezoidal(x, 2, 4, 8, 10), label='Trapezoidal')
# Sigmoid
plt.plot(x, sigmoid(x, 1, 5), label='Sigmoid')
# Generalized Bell
plt.plot(x, bell(x, 1, 2, 5), label='Generalized Bell')
# Gaussian
plt.plot(x, gaussian(x, 5, 1), label='Gaussian')

plt.title('Fuzzy Membership Functions')
plt.xlabel('x')
plt.ylabel('Membership')
plt.legend()
plt.grid(True)
plt.show()
```



QUESTION 3

Implement fuzzification on crisp set using following membership functions: i. Triangular Membership function ii. Trapezoidal membership function iii. Sigmoid membership function iv. Generalized bell membership function v. Gaussian membership function

```
import numpy as np

# Triangular Membership function
def triangular(x, a, b, c):
    if a <= x < b:
        return (x - a) / (b - a)
    elif b <= x < c:
        return (c - x) / (c - b)
    else:
        return 0

# Trapezoidal membership function
def trapezoidal(x, a, b, c, d):
    if a <= x < b:
        return (x - a) / (b - a)
    elif b <= x <= c:
        return 1
    elif c < x <= d:
        return (d - x) / (d - c)
    else:
        return 0
```

```

        return 0

# Sigmoid membership function
def sigmoid(x, a, c):
    return 1 / (1 + np.exp(-a * (x - c)))

# Generalized bell membership function
def generalized_bell(x, a, b, c):
    return 1 / (1 + np.abs((x - c) / a) ** (2 * b))

# Gaussian membership function
def gaussian(x, mean, sigma):
    return np.exp(-0.5 * ((x - mean) / sigma) ** 2)

# Crisp input
crisp_value = 0.7

# Fuzzification using various membership functions
triangular_membership = triangular(crisp_value, 0.2, 0.5, 0.8)
trapezoidal_membership = trapezoidal(crisp_value, 0.2, 0.4, 0.6, 0.8)
sigmoid_membership = sigmoid(crisp_value, 2, 0.5)
bell_membership = bell(crisp_value, 1, 2, 0.5)
gaussian_membership = gaussian(crisp_value, 0.5, 0.1)

print("Fuzzification using various membership functions:")
print("Triangular Membership:", triangular_membership)
print("Trapezoidal Membership:", trapezoidal_membership)
print("Sigmoid Membership:", sigmoid_membership)
print("Generalized Bell Membership:", bell_membership)
print("Gaussian Membership:", gaussian_membership)

Fuzzification using various membership functions:
Triangular Membership: 0.3333333333333336
Trapezoidal Membership: 0.5000000000000002
Sigmoid Membership: 0.598687660112452
Generalized Bell Membership: 0.998402555910543
Gaussian Membership: 0.1353352832366128

```

QUESTION 4

Realization of characteristics of fuzzy set by finding the following values: i. Core of a fuzzy set ii. Support of a fuzzy set. iii. Crossover points of a fuzzy set. iv. Bandwidth of a fuzzy set. v. Convexity of a fuzzy set.

```

import numpy as np

# Core of a fuzzy set
def core(fuzzy_set, threshold=0.5):
    return [x for x, y in enumerate(fuzzy_set) if y >= threshold]

# Support of a fuzzy set
def support(fuzzy_set, threshold=0.5):
    return [x for x, y in enumerate(fuzzy_set) if y > threshold]

# Crossover points of a fuzzy set
def crossover_points(fuzzy_set, threshold=0.5):
    points = []
    for i in range(len(fuzzy_set) - 1):
        if fuzzy_set[i] <= threshold < fuzzy_set[i + 1]:
            points.append(i + (threshold - fuzzy_set[i]) / (fuzzy_set[i + 1] - fuzzy_set[i]))
        elif fuzzy_set[i] > threshold >= fuzzy_set[i + 1]:
            points.append(i + 1 - (fuzzy_set[i] - threshold) / (fuzzy_set[i] - fuzzy_set[i + 1]))
    return points

# Bandwidth of a fuzzy set
def bandwidth(fuzzy_set):
    return len(fuzzy_set) - 1

# Convexity of a fuzzy set
def convexity(fuzzy_set):
    if all(fuzzy_set[i] <= fuzzy_set[i + 1] for i in range(len(fuzzy_set) - 1)):
        return "Convex"
    elif all(fuzzy_set[i] >= fuzzy_set[i + 1] for i in range(len(fuzzy_set) - 1)):
        return "Concave"
    else:
        return "Mixed"

# Example fuzzy set
fuzzy_set = [0.1, 0.3, 0.7, 1, 0.8, 0.5]

# Core of the fuzzy set
print("Core of the fuzzy set:", core(fuzzy_set))

# Support of the fuzzy set
print("Support of the fuzzy set:", support(fuzzy_set))

```

```

# Crossover points of the fuzzy set
print("Crossover points of the fuzzy set:", crossover_points(fuzzy_set))

# Bandwidth of the fuzzy set
print("Bandwidth of the fuzzy set:", bandwidth(fuzzy_set))

# Convexity of the fuzzy set
print("Convexity of the fuzzy set:", convexity(fuzzy_set))

```

```

Core of the fuzzy set: [2, 3, 4, 5]
Support of the fuzzy set: [2, 3, 4]
Crossover points of the fuzzy set: [1.5, 4.0]
Bandwidth of the fuzzy set: 5
Convexity of the fuzzy set: Mixed

```

QUESTION 6

Create fuzzy relation by Cartesian product of any two fuzzy sets and perform max-min and max-product composition on any two fuzzy relations.

```

import numpy as np
import skfuzzy as fuzz

# Define universe and membership functions for fuzzy sets A and B
x = np.arange(0, 11, 1)
A = fuzz.trimf(x, [3, 6, 9]) # Triangular membership function for fuzzy
set A
B = fuzz.trimf(x, [1, 4, 7]) # Triangular membership function for fuzzy
set B

# Create fuzzy relations based on the Cartesian product of A and B
relation_max_min = np.maximum.outer(A, B) # Max-min composition
relation_max_product = np.multiply.outer(A, B) # Max-product composition

print("Fuzzy Relation (Max-Min Composition):\n", relation_max_min)
print("\nFuzzy Relation (Max-Product Composition):\n",
relation_max_product)

```

```

Fuzzy Relation (Max-Min Composition):
[[0. 0. 0.33333333 0.66666667 1. 0.66666667
 0.33333333 0. 0. 0. 0.
[0. 0. 0.33333333 0.66666667 1. 0.66666667
 0.33333333 0. 0. 0. 0.
[0. 0. 0.33333333 0.66666667 1. 0.66666667
 0.33333333 0. 0. 0. 0.
[0. 0. 0.33333333 0.66666667 1. 0.66666667
 0.33333333 0. 0. 0. 0.

```

```

[0.33333333 0.33333333 0.33333333 0.66666667 1.          0.66666667
 0.33333333 0.33333333 0.33333333 0.33333333 0.33333333]
[0.66666667 0.66666667 0.66666667 0.66666667 1.          0.66666667
 0.66666667 0.66666667 0.66666667 0.66666667 0.66666667]
[1.          1.          1.          1.          1.          1.
 1.          1.          1.          1.          1.          ]
[0.66666667 0.66666667 0.66666667 0.66666667 1.          0.66666667
 0.66666667 0.66666667 0.66666667 0.66666667 0.66666667]
[0.33333333 0.33333333 0.33333333 0.66666667 1.          0.66666667
 0.33333333 0.33333333 0.33333333 0.33333333 0.33333333]
[0.          0.          0.33333333 0.66666667 1.          0.66666667
 0.33333333 0.          0.          0.          0.          ]
[0.          0.          0.33333333 0.66666667 1.          0.66666667
 0.33333333 0.          0.          0.          0.          ]]

```

Fuzzy Relation (Max-Product Composition):

```

[[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.11111111 0.22222222 0.33333333 0.22222222
 0.11111111 0.          0.          0.          0.          ]
[0.          0.          0.22222222 0.44444444 0.66666667 0.44444444
 0.22222222 0.          0.          0.          0.          ]
[0.          0.          0.33333333 0.66666667 1.          0.66666667
 0.33333333 0.          0.          0.          0.          ]
[0.          0.          0.22222222 0.44444444 0.66666667 0.44444444
 0.22222222 0.          0.          0.          0.          ]
[0.          0.          0.11111111 0.22222222 0.33333333 0.22222222
 0.11111111 0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          ]]]

```

QUESTION 7

Implementation of fuzzy distance metrics: i. Fuzzy Euclidian distance ii. Fuzzy hamming distance

```

# Fuzzy Euclidian Distance

def euclidian_distance(a,b):
    distance = [(i-j)**2 for i, j in zip(a,b)]
    return (sum(distance))**0.5
a = [0.3, 0.8, 0.1]
b = [0.5, 0.7, 0.2]
print("EUCLIDIAN DISTANCE OF a, b", euclidian_distance(a,b))

```



```
# Fuzzy Hamming Distance

def hamming_distance(a,b):
    distance = [(i-j) for i, j in zip(a,b)]
    return (sum(distance))
a = [0.3, 0.8, 0.1]
b = [0.5, 0.7, 0.2]
print("HAMMING DISTANCE OF a, b", hamming_distance(a,b))
EUCLIDIAN DISTANCE OF a, b 0.24494897427831785
HAMMING DISTANCE OF a, b -0.19999999999999993
```

QUESTION 9

Implement a program that performs fuzzy clustering on a set of data points and outputs the membership degrees for each data point in each cluster.

```
import numpy as np
import skfuzzy as fuzz

# Generate sample data
np.random.seed(42)
data = np.random.randn(100, 2)

# Perform fuzzy clustering (FCM algorithm)
centers, u, _, _, _, _ = fuzz.cluster.cmeans(
    data.T, 3, m=2, error=0.005, maxiter=1000
)

# data.T transposes the data to match the expected input format.
# 3 specifies the number of clusters.
# m=2 sets the fuzziness parameter.
# error and maxiter are stopping criteria for the algorithm.

# Output membership degrees for each data point in each cluster
print("Membership degrees:")
print(u)
#u: A matrix where each row represents a data point, and each column
represents the membership degree of that data point to each cluster.
#We print the membership degrees (u) for each data point in each cluster.
Membership degrees:
[[5.27217810e-01 4.82916479e-02 3.39511520e-01 2.16324636e-01
 9.47691322e-02 3.26388325e-01 7.7759822e-01 1.81236231e-01
 2.72193799e-02 5.08252714e-01 4.88279141e-01 8.87675928e-01
 6.07940472e-02 4.39673700e-02 1.28623735e-01 1.09931842e-01
```

9.39948350e-01	8.64524076e-01	7.65052399e-01	5.06386050e-02
3.08786227e-01	4.75507819e-01	2.05099364e-01	8.36711462e-02
8.18315807e-01	7.31093453e-01	7.93854714e-02	9.66586566e-02
6.55824565e-02	7.12089163e-04	1.45838185e-01	3.97113086e-01
5.25527155e-02	3.66361078e-02	9.16201753e-01	3.77863665e-02
5.48483101e-02	1.82346028e-01	5.90507575e-01	7.48952918e-01
1.40081095e-01	5.76905046e-01	1.65989852e-01	2.41307894e-01
8.68666481e-02	1.52973407e-02	1.04958482e-01	7.25744519e-01
2.19898981e-01	4.94018415e-01	1.24155390e-01	6.68019395e-01
1.37833648e-01	3.62839603e-01	4.48755931e-01	1.38719834e-01
1.21673667e-01	1.55173361e-01	9.18776328e-01	1.45573853e-01
8.72377879e-01	7.05940277e-01	1.04879718e-01	1.64316331e-01
7.80087961e-01	8.41345106e-02	5.08797639e-02	1.17615109e-01
8.03512546e-02	8.66237586e-01	1.67152653e-02	9.09213160e-02
2.05679948e-02	3.96738434e-01	2.13415560e-01	1.71393909e-01
3.34158200e-02	9.51829445e-01	2.99856954e-01	7.84522537e-02
8.67742481e-02	7.31737238e-01	2.11502092e-01	9.22869121e-02
7.26054203e-01	3.17175711e-01	1.61134496e-01	1.31385140e-02
4.22560233e-02	1.40969755e-01	9.22106145e-01	5.24092574e-02
8.51714553e-02	4.72597484e-01	5.43245985e-01	8.87077771e-02
9.55679355e-01	1.51887273e-01	5.70010708e-03	9.52771364e-01]
[1.96231580e-01	6.63855938e-02	5.23585598e-01	1.53468301e-01
6.05443492e-01	5.85766731e-01	1.40179889e-01	7.08666011e-01
9.34740757e-01	3.84619051e-01	1.72246034e-01	7.48995167e-02
8.71499069e-01	8.92882481e-01	8.12763343e-01	2.80026766e-01
4.17667445e-02	7.31954845e-02	1.48786747e-01	8.92665964e-01
1.73565259e-01	3.85367288e-01	6.94852705e-01	3.10962992e-01
1.12885770e-01	1.47270663e-01	6.98393439e-01	8.93791876e-02
9.00472741e-01	1.12003125e-03	7.75644255e-01	4.93465545e-01
6.39370872e-02	9.00467857e-02	4.86118012e-02	6.15340683e-02
1.15058179e-01	5.78946063e-01	2.58454186e-01	1.61823234e-01
4.96343347e-01	1.62784727e-01	7.72154543e-01	1.50030746e-01
6.73187839e-01	3.09260071e-02	8.47067370e-01	2.00301630e-01
2.24278585e-01	3.34514352e-01	8.01036373e-01	2.63129930e-01
4.19828807e-01	1.83862310e-01	2.63949573e-01	7.35053531e-01
2.09846960e-01	4.98656841e-01	5.64689931e-02	1.15762626e-01
6.58629767e-02	1.44244560e-01	1.53667806e-01	7.69620700e-01
1.43937924e-01	8.34131839e-01	8.63330323e-01	3.92626807e-01
8.80370678e-01	7.25390168e-02	2.98643783e-02	8.11563889e-01
3.30684935e-02	4.84761399e-01	1.69873879e-01	2.08590775e-01
9.20182938e-01	2.92881773e-02	1.77760767e-01	7.71287128e-01
6.89056843e-01	1.21783353e-01	1.39991846e-01	1.21909273e-01
2.11626988e-01	5.95318107e-01	3.84855925e-01	2.10237415e-02
8.76486777e-02	2.55644770e-01	4.21200440e-02	8.58663714e-01
2.84464170e-01	2.09273627e-01	3.50023916e-01	3.84857548e-01
2.85846472e-02	2.19578417e-01	9.87826234e-01	3.19892555e-02]
[2.76550610e-01	8.85322758e-01	1.36902882e-01	6.30207063e-01
2.99787376e-01	8.78449443e-02	8.20602894e-02	1.10097757e-01
3.80398627e-02	1.07128235e-01	3.39474825e-01	3.74245554e-02
6.77068836e-02	6.31501495e-02	5.86129226e-02	6.10041392e-01
1.82849050e-02	6.22804396e-02	8.61608544e-02	5.66954307e-02
5.17648514e-01	1.39124892e-01	1.00047931e-01	6.05365862e-01
6.87984231e-02	1.21635884e-01	2.22221090e-01	8.13962156e-01
3.39448027e-02	9.98167880e-01	7.85175596e-02	1.09421370e-01

```

8.83510197e-01 8.73317107e-01 3.51864455e-02 9.00679565e-01
8.30093511e-01 2.38707909e-01 1.51038239e-01 8.92238480e-02
3.63575558e-01 2.60310226e-01 6.18556050e-02 6.08661360e-01
2.39945513e-01 9.53776652e-01 4.79741477e-02 7.39538509e-02
5.55822434e-01 1.71467233e-01 7.48082372e-02 6.88506749e-02
4.42337545e-01 4.53298087e-01 2.87294496e-01 1.26226635e-01
6.68479373e-01 3.46169798e-01 2.47546792e-02 7.38663521e-01
6.17591443e-02 1.49815162e-01 7.41452476e-01 6.60629686e-02
7.59741151e-02 8.17336508e-02 8.57899133e-02 4.89758084e-01
3.92780677e-02 6.12233974e-02 9.53420356e-01 9.75147948e-02
9.46363512e-01 1.18500167e-01 6.16710561e-01 6.20015315e-01
4.64012416e-02 1.88823775e-02 5.22382280e-01 1.50260618e-01
2.24168909e-01 1.46479410e-01 6.48506061e-01 7.85803815e-01
6.23188092e-02 8.75061824e-02 4.54009579e-01 9.65837745e-01
8.70095299e-01 6.03385475e-01 3.57738105e-02 8.89270283e-02
6.30364375e-01 3.18128889e-01 1.06730099e-01 5.26434675e-01
1.57359975e-02 6.28534310e-01 6.47365925e-03 1.52393803e-02]]

```

QUESTION 10

Implement and plot different defuzzification methods: i. α - cut and strong α - cuT ii. Maxima method: a. First of Maxima (FOM) b. Last of Maxima (LOM) c. Mean of Maxima (MOM) iii. Centroid method: a. Center of Sums (COS) b. Center of gravity (COG) / Centroid of Area (COA) Method c. Weighted average method

```

import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz

# Generate sample fuzzy set
x = np.linspace(0, 10, 1000)
mfx = fuzz.trapmf(x, [2, 4, 6, 8]) # Sample fuzzy set (trapezoidal shape)

# Plot the sample fuzzy set
plt.figure(figsize=(10, 5))
plt.plot(x, mfx, 'b', linewidth=1.5, label='Fuzzy Set')
plt.fill_between(x, 0, mfx, alpha=0.1)

# Defuzzification methods
#  $\alpha$ -cut and strong  $\alpha$ -cut (calculate manually)
alpha = 0.5
alpha_cut = np.min(x[mfx >= alpha]) #  $\alpha$ -cut for  $\alpha=0.5$ 
strong_alpha_cut = np.max(x[mfx >= alpha]) # Strong  $\alpha$ -cut for  $\alpha=0.5$ 

# Maxima methods
fom = fuzz.defuzz(x, mfx, 'lom') # First of Maxima
lom = fuzz.defuzz(x, mfx, 'lom') # Last of Maxima
mom = fuzz.defuzz(x, mfx, 'centroid') # Mean of Maxima

```

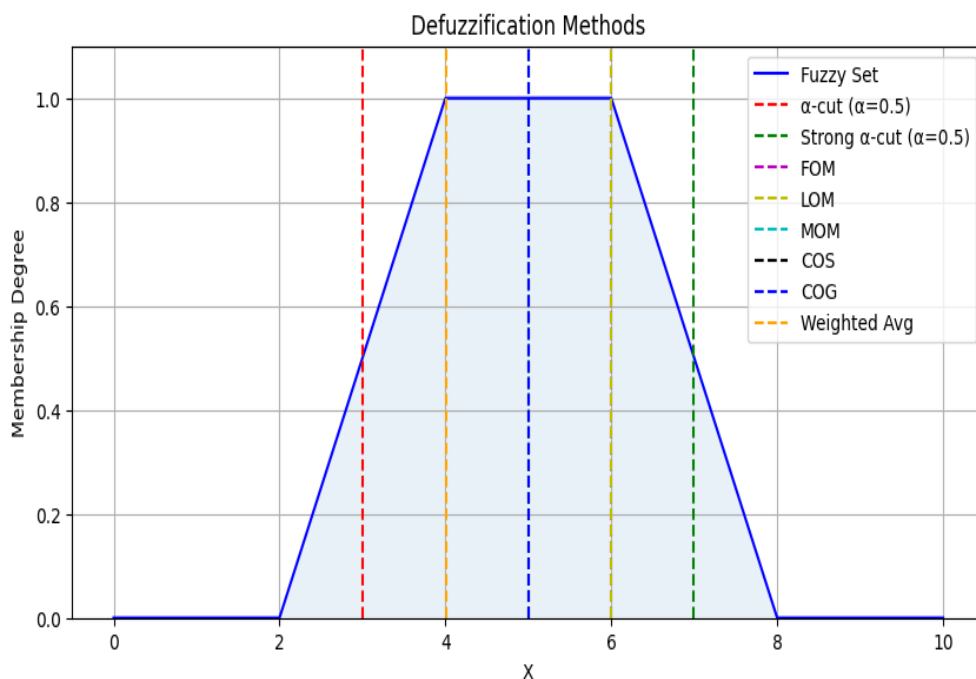
```

# Centroid methods
cos = fuzz.defuzz(x, mfx, 'centroid') # Center of Sums
cog = fuzz.defuzz(x, mfx, 'centroid') # Center of gravity / Centroid of Area
weighted_avg = fuzz.defuzz(x, mfx, 'som') # Weighted average method

# Plot defuzzified values
plt.axvline(x=alpha_cut, color='r', linestyle='--', label='α-cut (α=0.5)')
plt.axvline(x=strong_alpha_cut, color='g', linestyle='--', label='Strong α-cut (α=0.5)')
plt.axvline(x=fom, color='m', linestyle='--', label='FOM')
plt.axvline(x=lom, color='y', linestyle='--', label='LOM')
plt.axvline(x=mom, color='c', linestyle='--', label='MOM')
plt.axvline(x=cos, color='k', linestyle='--', label='COS')
plt.axvline(x=cog, color='b', linestyle='--', label='COG')
plt.axvline(x=weighted_avg, color='orange', linestyle='--', label='Weighted Avg')

# Additional plotting configurations
plt.ylim(0, 1.1)
plt.title('Defuzzification Methods')
plt.xlabel('X')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid(True)
plt.show()

```



QUESTION 11

Implement the AND, OR and XOR function using single layer and multilayer neural network.

```
# Implementation of the single-layer neural network:

import numpy as np
import tensorflow as tf

# Define the inputs and outputs for AND, OR, and XOR functions
inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
and_outputs = np.array([[0], [0], [0], [1]], dtype=np.float32)
or_outputs = np.array([[0], [1], [1], [1]], dtype=np.float32)
xor_outputs = np.array([[0], [1], [1], [0]], dtype=np.float32)

# Define the single-layer neural network
def single_layer_nn(inputs, outputs):
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(units=1, input_shape=(2,)),
        activation='sigmoid'
    ])
    model.compile(optimizer='adam', loss='mse')
    model.fit(inputs, outputs, epochs=1000, verbose=0)
    return model

# Train single-layer neural network for AND function
and_model = single_layer_nn(inputs, and_outputs)
print("AND function predictions:")
print(and_model.predict(inputs))

# Train single-layer neural network for OR function
or_model = single_layer_nn(inputs, or_outputs)
print("\nOR function predictions:")
print(or_model.predict(inputs))

# Train single-layer neural network for XOR function
xor_model = single_layer_nn(inputs, xor_outputs)
print("\nXOR function predictions:")
print(xor_model.predict(inputs))
```

AND function predictions:

```
1/1 [=====] - 0s 74ms/step
[[0.2864885]
 [0.2833026]
 [0.47743544]]
```

```
[0.47353533]]
```

OR function predictions:

```
1/1 [=====] - 0s 62ms/step  
[[0.5915293 ]  
 [0.84040105]  
 [0.6308142 ]  
 [0.8613604 ]]
```

XOR function predictions:

```
1/1 [=====] - 0s 43ms/step  
[[0.43559954]  
 [0.44772026]  
 [0.53782403]  
 [0.5500172 ]]
```

```
# Implementation of the multi-layer neural network:  
  
# Define the multi-layer neural network  
def multi_layer_nn(inputs, outputs):  
    model = tf.keras.Sequential([  
        tf.keras.layers.Dense(units=2, input_shape=(2,)),  
activation='sigmoid'),  
        tf.keras.layers.Dense(units=1, activation='sigmoid')  
    ])  
    model.compile(optimizer='adam', loss='mse')  
    model.fit(inputs, outputs, epochs=1000, verbose=0)  
    return model  
  
# Train multi-layer neural network for AND function  
and_model_multi = multi_layer_nn(inputs, and_outputs)  
print("\nAND function predictions (multi-layer NN):")  
print(and_model_multi.predict(inputs))  
  
# Train multi-layer neural network for OR function  
or_model_multi = multi_layer_nn(inputs, or_outputs)  
print("\nOR function predictions (multi-layer NN):")  
print(or_model_multi.predict(inputs))  
  
# Train multi-layer neural network for XOR function  
xor_model_multi = multi_layer_nn(inputs, xor_outputs)  
print("\nXOR function predictions (multi-layer NN):")  
print(xor_model_multi.predict(inputs))
```

AND function predictions (multi-layer NN):

```
1/1 [=====] - 0s 87ms/step  
[[0.3075561 ]  
 [0.3149789 ]  
 [0.32447037]
```

[0.32506394]]

OR function predictions (multi-layer NN):

1/1 [=====] - 0s 52ms/step

[[0.65141016]

[0.6440638]

[0.6673036]

[0.6645457]]

XOR function predictions (multi-layer NN):

1/1 [=====] - 0s 55ms/step

[[0.50139815]

[0.47056708]

[0.52781355]

[0.4912265]]